UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

# Programación III

## Tarea #5: Patrones de Diseño

**Pregrado**

**2026-2**

**Profesor: José A. Chávez Álvarez**

**Lab 1.11**

## Indicaciones específicas:

- Esta evaluación contiene 6 páginas.
- La solución de la tarea deber ser entregada con un link al repositorio en GitHub.

# Tarea #5: Patrones de Diseño

## Ejercicio 1: Singleton - Transaction Manager Concurrente

Diseñe un sistema de gestión de transacciones bancarias donde múltiples módulos puedan registrar operaciones sobre cuentas. Todas las operaciones deben ser coordinadas por una única instancia global de un administrador transaccional.

### Requisitos

Debe implementar obligatoriamente:

- Clase `TransactionManager` (Singleton).
- Clase `Transaction`.
- El `TransactionManager` debe exponer:
  - `deposit(Account*, double)`
  - `withdraw(Account*, double)`

Debe garantizarse que las operaciones sean atómicas.

### Simulación mínima esperada

```
int main() {

    auto& manager = TransactionManager::getInstance();

    Account account("A1", 1000);

    std::vector<std::thread> threads;

    for (int i = 0; i < 10; ++i) {
        threads.emplace_back([&]() {
            manager.deposit(&account, 100);
            manager.withdraw(&account, 50);
```

```
        });
    }

    for (auto& t : threads) t.join();

    std::cout << "Balance final: "
              << account.getBalance()
              << std::endl;

    std::cout << "Direccion manager: "
              << &TransactionManager::getInstance()
              << std::endl;
}
```

### Resultado Esperado

- El balance final debe ser 1000 - (10 * 50).
- No deben existir condiciones de carrera.
- La dirección del `TransactionManager` debe ser única.

## Ejercicio 2: Abstract Factory - Motor Multiplataforma

Diseñe un sistema de renderizado que permita cambiar dinámicamente entre OpenGL y Vulkan sin modificar el código cliente.

**Clases obligatorias**

- Interfaz `RenderFactory`
- `OpenGLFactory`
- `VulkanFactory`
- `Window`
- `Button`
- `Shader`

**Restricciones**

- El cliente solo conoce `RenderFactory`.
- No se permite usar `if` o `switch` para seleccionar plataforma.
- Cada producto debe tener comportamiento distinto según plataforma.

**Problema específico**

El sistema debe permitir cambiar la fábrica en tiempo de ejecución y crear nuevos objetos compatibles con la nueva plataforma.

**Validación mínima**

```
int main() {
    std::unique_ptr<RenderFactory> factory =
        std::make_unique<OpenGLFactory>();

    auto window = factory->createWindow();
    auto button = factory->createButton();

    window->draw();
    button->draw();

    factory = std::make_unique<VulkanFactory>();

    auto shader = factory->createShader();
    shader->compile();
}
```

# Ejercicio 3: Composite - Expresiones Matemáticas

Diseñe un sistema que represente expresiones matemáticas simbólicas y permita evaluarlas y derivarlas.

**Clases obligatorias**

- Clase abstracta `Expression`
- `Constant`
- `Variable`
- `BinaryOperation`
- `UnaryFunction`

**Métodos obligatorios**

- `double evaluate(double x)`
- `std::shared_ptr<Expression> derivative()`
- `std::string toString()`

**Restricciones**

- No se permite `dynamic_cast`.

- La derivación debe respetar reglas matemáticas reales.
- Debe permitir anidamiento arbitrario.

**Problema específico**
Representar y derivar la expresión:

$$f(x) = (x^2 + 3x) * \sin(x)$$

## Validación mínima

```
int main() {
    auto x = std::make_shared<Variable>();

    auto expr =
        std::make_shared<Multiply>(
            std::make_shared<Add>(
                std::make_shared<Power>(
                    x,
                    std::make_shared<Constant>(2)
                ),
                std::make_shared<Multiply>(
                    std::make_shared<Constant>(3),
                    x
                )
            ),
            std::make_shared<Sin>(x)
        );

    auto deriv = expr->derivative();

    std::cout << "f(x) = " << expr->toString() << std::endl;
    std::cout << "f'(x) = " << deriv->toString() << std::endl;

    std::cout << "f(2) = " << expr->evaluate(2) << std::endl;
}
```

# Ejercicio 4: Iterator - Recorrido de Grafo

Implemente un grafo dirigido y un iterador personalizado que permita recorrerlo en BFS y DFS.

**Clases obligatorias**

- `Graph`
- `Node`

- `GraphIterator` (interfaz)
- `BFSIterator`
- `DFSIterator`

**Restricciones**

- No exponer la estructura interna.
- Evitar ciclos infinitos.
- El iterador debe poder cambiarse dinámicamente.

**Validación mínima**

```
int main() {
    Graph g;
    g.addEdge(1,2);
    g.addEdge(1,3);
    g.addEdge(2,4);

    std::unique_ptr<GraphIterator> it =
        std::make_unique<BFSIterator>(g, 1);

    while(it->hasNext()) {
        std::cout << it->next() << " ";
    }
}
```